

1. Введение

В современной среде кибербезопасности сценарии PowerShell стали одним из основных инструментов реализации атак типа «Living off the Land» (LotL). Благодаря глубокой интеграции с операционной системой Windows и доступу к низкоуровневым API, PowerShell позволяет злоумышленникам выполнять вредоносный код в оперативной памяти, минуя традиционные файловые антивирусные сканеры.

Данный проект посвящен разработке интеллектуальной системы детекции вредоносных PowerShell-скриптов с применением рекуррентных нейронных сетей архитектуры Bidirectional LSTM. Работа выполнена в рамках курса «Прикладные методы искусственного интеллекта».

2. Постановка задачи и анализ предметной области

2.1 Актуальность темы

Традиционные сигнатурные методы обнаружения вредоносного ПО малоэффективны против современных PowerShell-угроз из-за активного использования техник обфускации: динамической генерации команд, кодирования полезной нагрузки в Base64 и использования случайных имен переменных.

2.2 Цель и задачи

Цель работы: Разработка классификатора, способного по текстовому содержимому скрипта определять его принадлежность к категории вредоносных (Malicious) или безопасных (Benign).

Основные задачи:

- Сбор и предварительный анализ данных (EDA).
- Разработка алгоритма очистки данных от индикаторов компрометации (IoC) для предотвращения переобучения.
- Построение и обучение нейросетевой модели BiLSTM.
- Интеграция системы мониторинга экспериментов ClearML.

3. Архитектура решения и подготовка данных

3.1 Описание набора данных

Для обучения модели был использован набор данных **MPSD_Dataset**. Состав обучающей выборки представлен в таблице ниже:

Training Data		
Type	Count	Label
malicious_pure	4202	1
mixed_malicious	4202	1
benign	4316	0

3.2 Предобработка и токенизация

Ключевой особенностью системы является кастомный пайплайн очистки данных. Функция `clean_data_to_tokens` выполняет следующие действия:

1. **Замена Base64-нагрузки:** Строки длиной более 50–100 символов заменяются токеном `[BASE64_PAYLOAD]`.
2. **Классификация IP-адресов:** Внутренние (127.0.0.1, 192.168.x.x) и внешние адреса заменяются на `[INTERNAL_IP]` и `[EXTERNAL_IP]` соответственно.
3. **Унификация значений:** Шестнадцатеричные числа (0x...) заменяются на `[HEX_VALUE]`, а длинные цифровые последовательности на `[LONG_NUMBER]`.
4. **Нормализация синтаксиса:** Удаление спецсимволов и приведение текста к нижнему регистру.

3.3 Разведочный анализ (EDA)

В ходе анализа распределения длин скриптов было выявлено, что подавляющее большинство файлов содержит менее 1000 слов. Для оптимизации вычислительных ресурсов и устранения влияния аномально длинных скриптов, максимальная длина последовательности была ограничена по **95-му перцентилю**. Это позволило ускорить время инференса до значения менее **50 мс** на один объект.

4. Проектирование модели нейронной сети

Для решения задачи классификации выбрана архитектура **Bidirectional LSTM**, реализованная на базе Keras/TensorFlow.

4.1 Параметры архитектуры

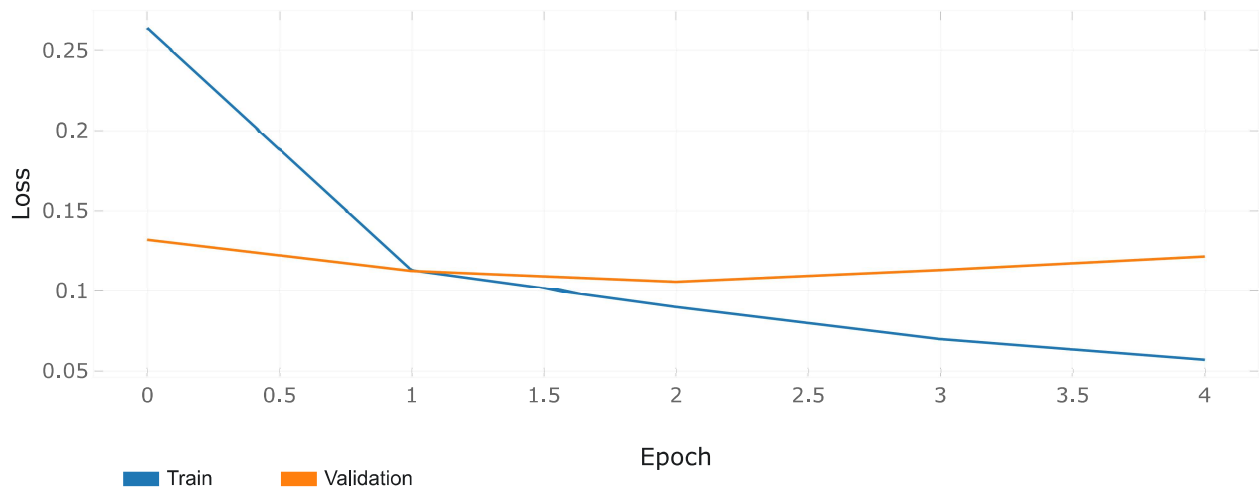
- **Входной уровень:** Слой `TextVectorization` с размером словаря в **90 000** токенов. Слой встроен непосредственно в модель, что гарантирует идентичность словарей при обучении и инференсе.
- **Слой эмбеддингов:** Размерность вектора — **32**. Используется параметр `mask_zero=True` для игнорирования заполняющих нулей (`padding`).
- **Рекуррентные слои:** Два двунаправленных слоя LSTM с количеством блоков **64** и **32** соответственно.
- **Регуляризация:** Применение слоев `Dropout` с коэффициентом **0.3** для предотвращения переобучения.
- **Выходной слой:** Слой `Dense` с 2 нейронами и функцией активации `softmax` для получения вероятностного распределения классов.

5. Оценка результатов обучения

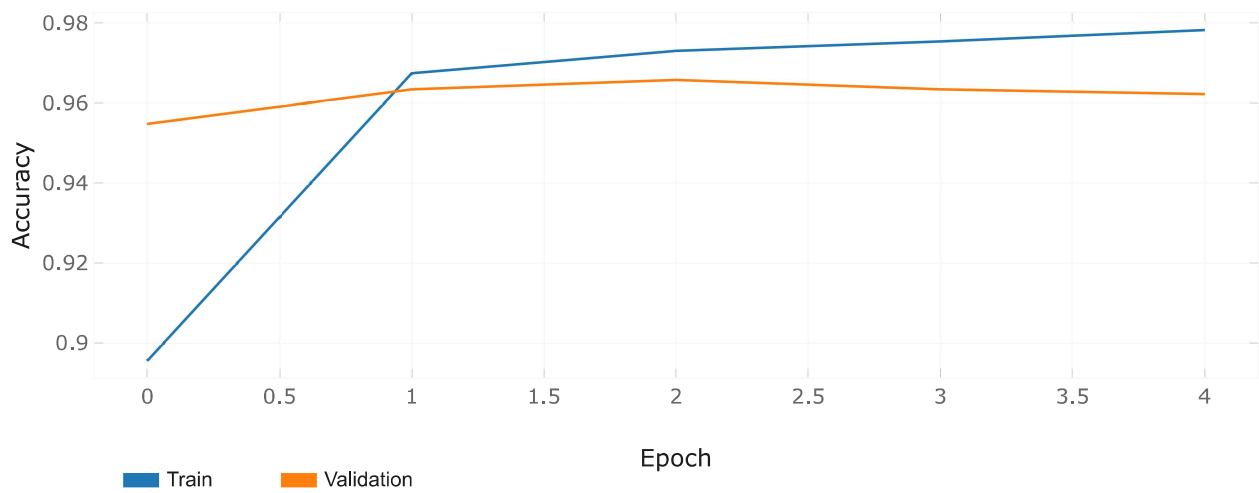
Процесс обучения контролировался через систему **ClearML**, что позволило отслеживать метрики в реальном времени.

5.1 Итоговые метрики

Loss

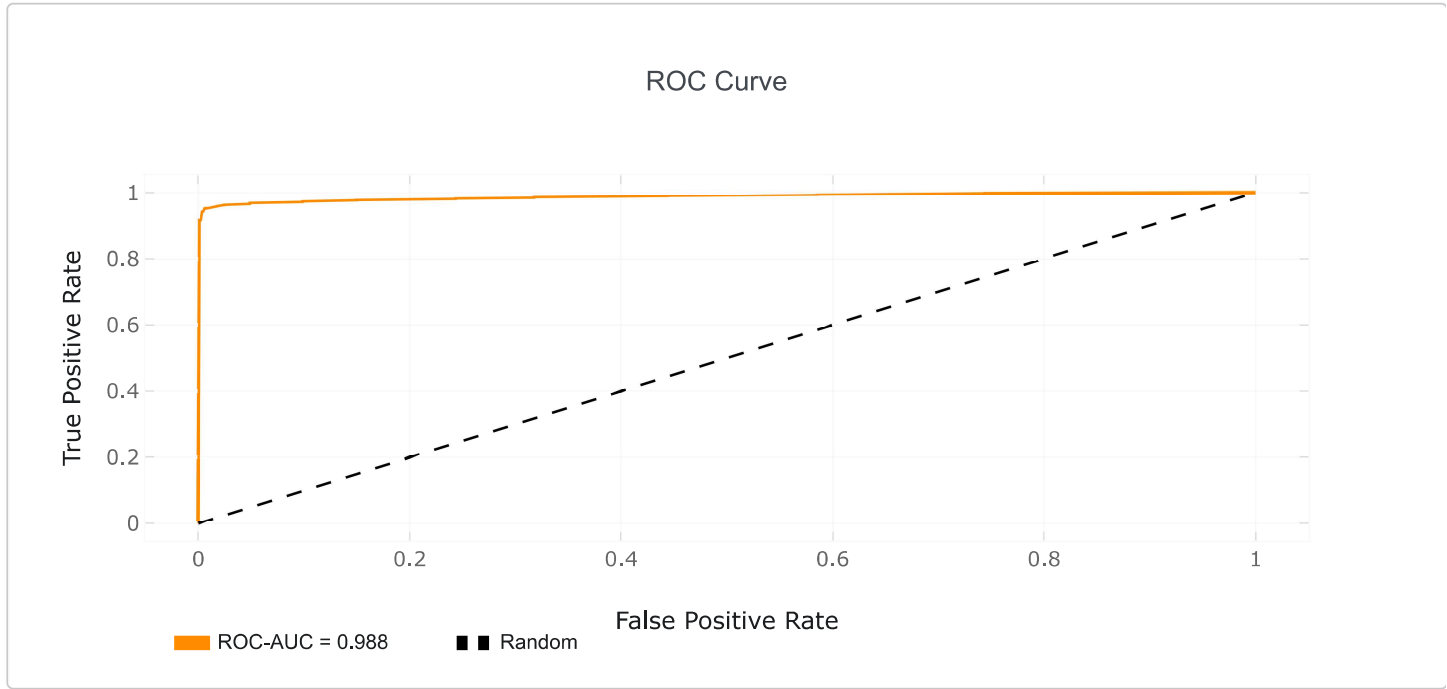


Accuracy

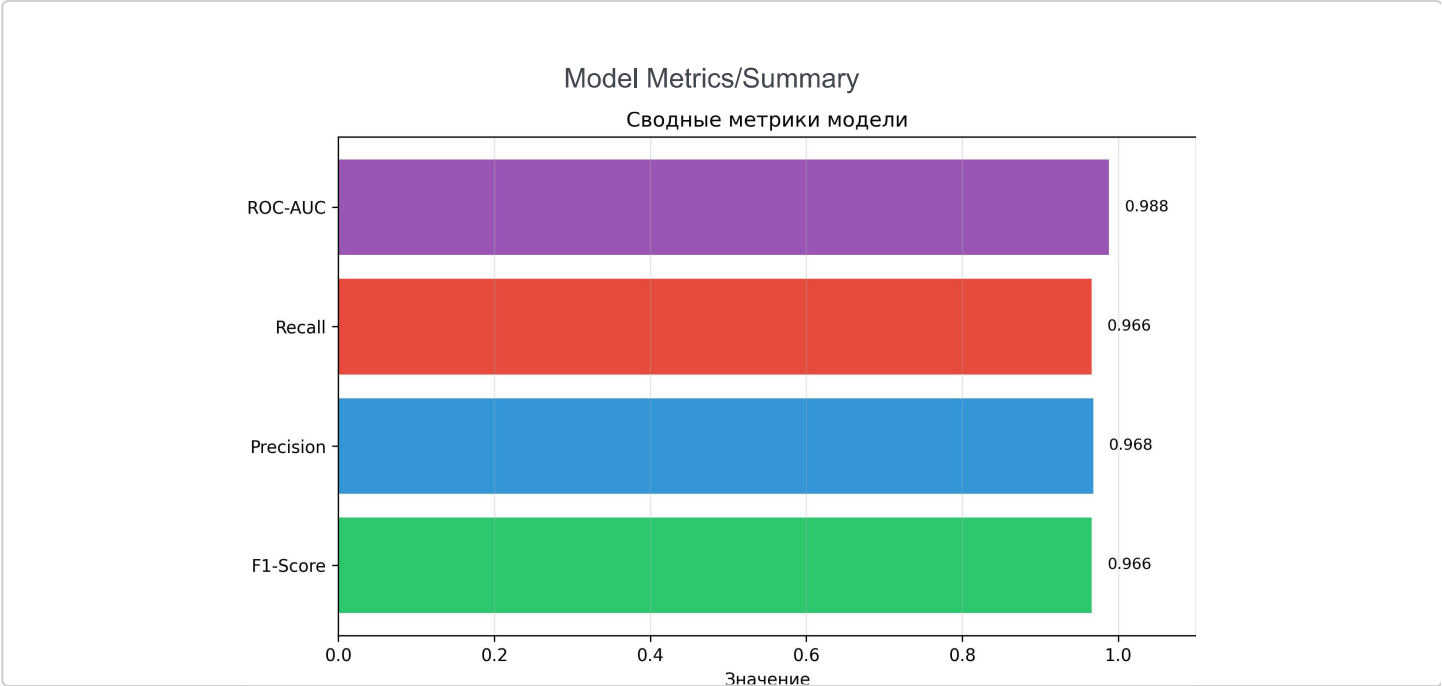


Комментарий к графикам: Модель демонстрирует стабильное обучение без явных признаков переобучения, кривые валидации и обучения сходятся корректно.

Площадь под кривой ошибок (ROC-AUC) демонстрирует высокую разделяющую способность модели.



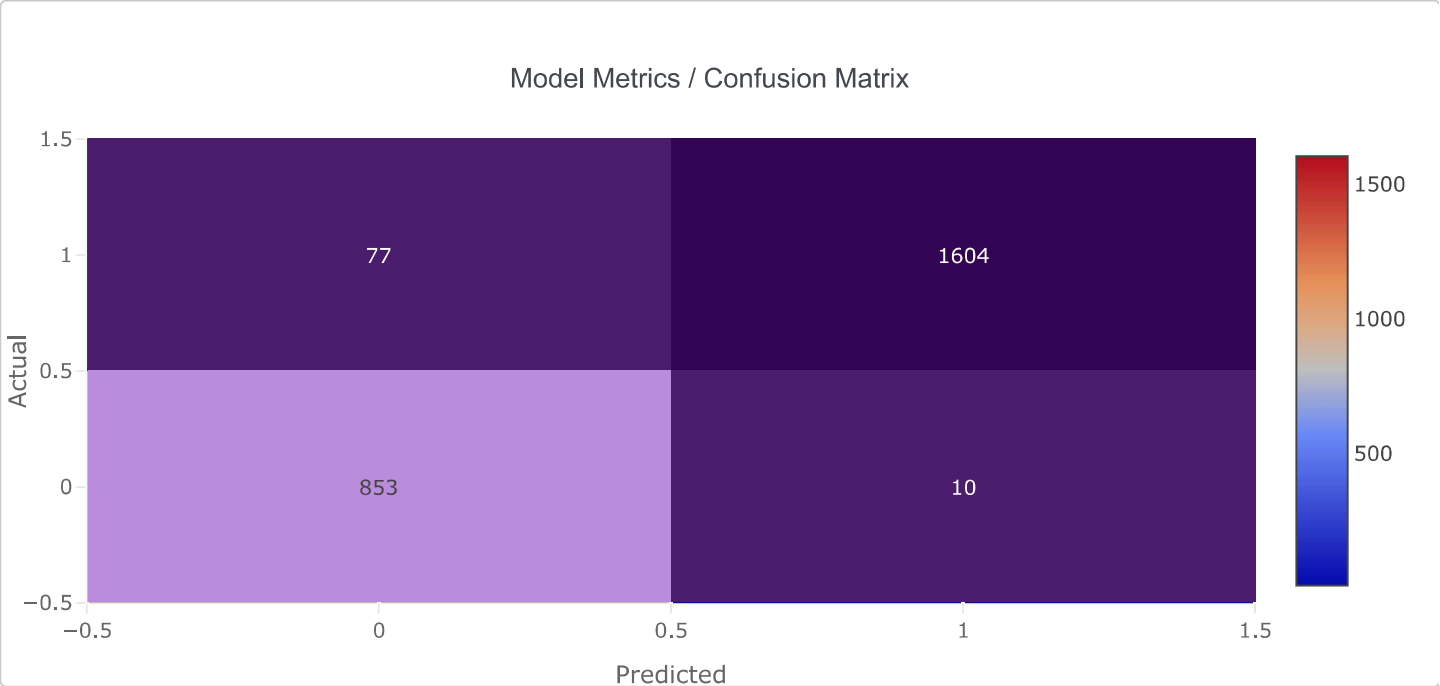
На валидационной выборке модель продемонстрировала следующие показатели:



5.2 Анализ матрицы ошибок (Confusion Matrix)

Согласно результатам валидации:

- **True Positives (1604):** Модель успешно идентифицировала большинство вредоносных сценариев.
- **False Positives (10):** Крайне низкий уровень ложноположительных срабатываний на безопасных скриптах, что критически важно для минимизации нагрузки на специалистов по ИБ.
- **False Negatives (77):** Некоторое количество вредоносных скриптов было классифицировано как безопасные, что обусловлено их структурной схожестью с легитимными инструментами администрирования.



6. Выводы

В рамках проекта был успешно разработан и обучен нейросетевой детектор вредоносных сценариев PowerShell.

1. Применение специализированной очистки данных и абстрактных мета-токенов позволило модели выучить логику работы вредоносного ПО, сделав ее устойчивой к базовой обфускации.
2. Внедрение слоя TextVectorization внутрь графа модели значительно упростило процесс инференса (predict.py).
3. Использование ClearML позволило выстроить прозрачный и легко воспроизводимый MLOps-процесс от подготовки данных до деплоя артефакта модели.